

Programação de Aplicativos Mobile II

Projeto ClimaTech Acesso à API OpenMeteo.com

- ☐ **API gratuitas e API Key**

- ☐ **Layout**

- ☐ **Arquivos do projeto**

 - ☐ AndroidManifest.xml

 - ☐ build.gradle.kts

 - ☐ **API**

 - ☐ ApiService.java

 - ☐ HttpResponse.java

 - ☐ RetrofitClient.java

 - ☐ **Models**

 - ☐ TempoAtual.java

 - ☐ MainActivity.java

 - ☐ Activity_main.xml

1. Open-Meteo (Recomendada)

Esta é uma das melhores opções gratuitas e de código aberto. 

- **Vantagens:** Não requer chave de API (API Key), cadastro ou cartão de crédito para uso não comercial.
- **Recursos:** Dados históricos e previsões de alta resolução.
- **Uso:** Ótima para protótipos e aplicações rápidas.  open-meteo.com +1

<https://open-meteo.com/>



2. OpenWeatherMap

Uma das mais populares e consolidadas no mercado.

- **Plano Gratuito:** Oferece previsões atuais, previsão por hora e diária.
- **Limites:** Geralmente permite um alto número de chamadas diárias (ex: 1.000.000/mês ou 60/minuto no plano *Free*), ideal para estudantes e projetos pequenos.
- **Recursos:** Dados de temperatura, umidade, vento, nascer/pôr do sol.  OpenWeather +3

<https://openweathermap.org/>



3. WeatherAPI

<https://www.weatherapi.com/>

Muito fácil de usar e com excelente documentação.

- **Plano Gratuito:** Disponibiliza condições atuais, previsão para 3 dias, geolocalização e alertas meteorológicos.
- **Recursos:** Ótima cobertura, incluindo dados de radar e previsões horárias.  Reddit · r/learnpython +2



4. HG Brasil Weather (Foco no Brasil)

<https://hgbrasil.com/weather>

Ótima opção para quem precisa de dados específicos e em português no Brasil.

- **Recursos:** Previsão detalhada, temperaturas máxima/mínima, umidade e vento para cidades brasileiras.
- **Vantagens:** Fácil integração (API GET), suporte a CORS.  HG Brasil



5. Visual Crossing

<https://www.visualcrossing.com/>

Focada em dados históricos e previsões de ponto único. 

- **Plano Gratuito:** Disponibiliza previsões de até 15 dias e histórico.
- **Vantagens:** Boa interface de construção de consulta para criar URLs da API.  Reddit · r/learnpython +4



6. Meteoblue

<https://www.meteoblue.com/pt/tempo/semana/>

Oferece alta precisão e grande variedade de dados. 

- **Plano Gratuito:** Permite até 5.000 chamadas de API por ano.
- **Recursos:** Mais de 100 variáveis meteorológicas diferentes, incluindo mapas e gráficos.  meteoblue +1





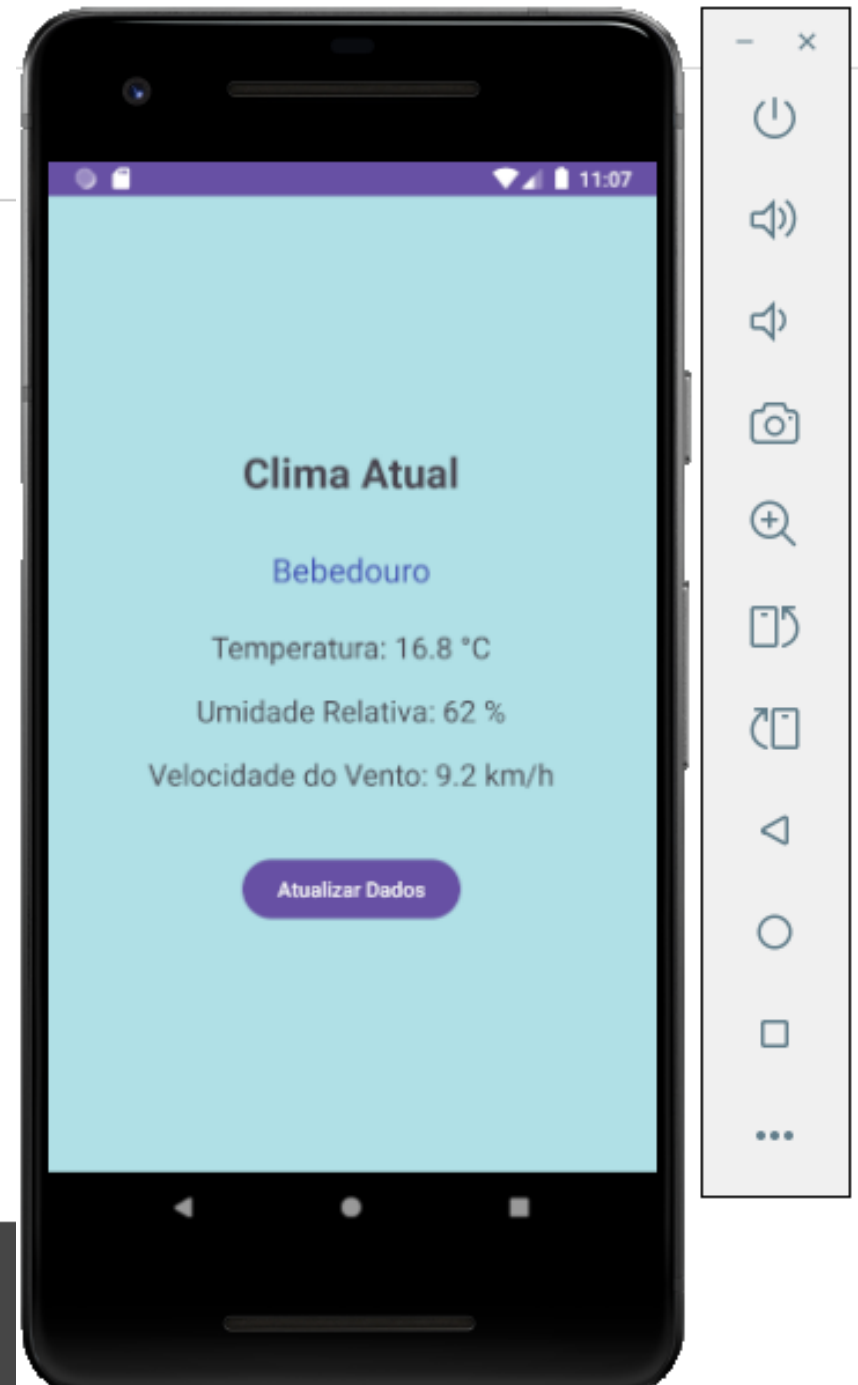
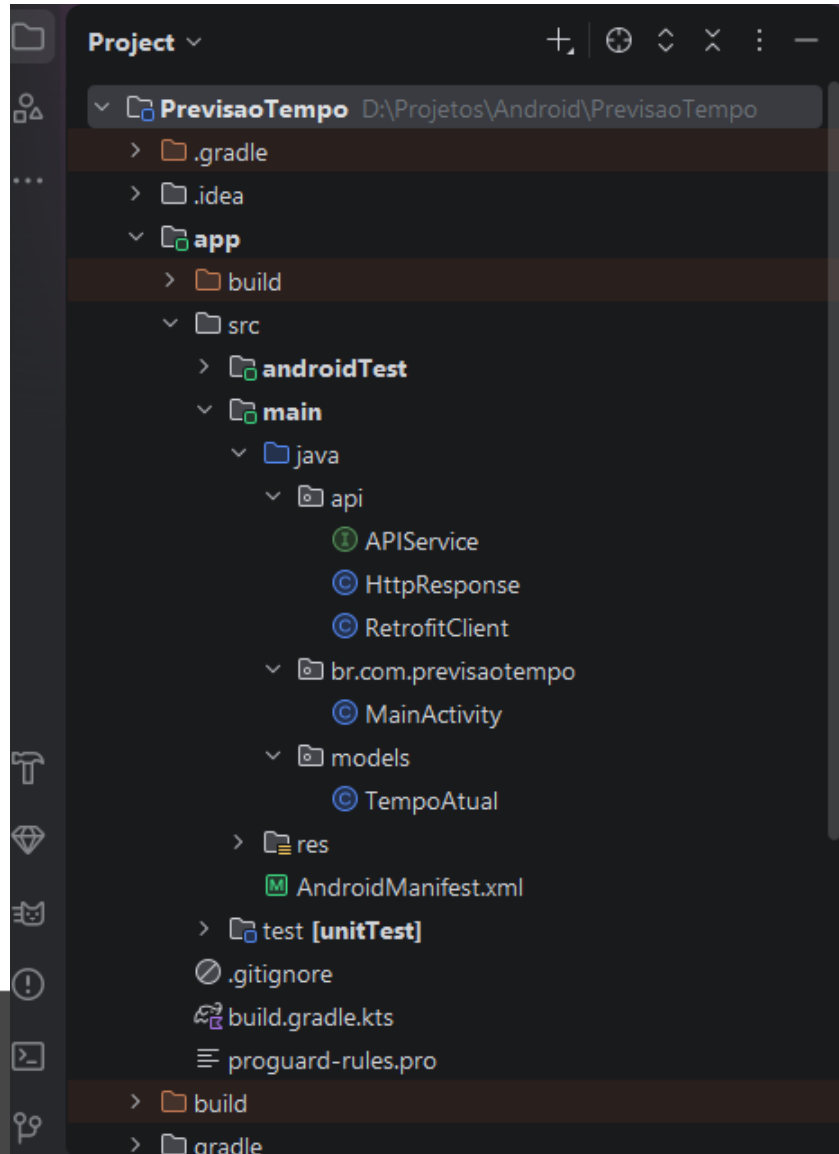
Free Weather API

Open-Meteo is an open-source weather API and offers free access for non-commercial use. No API key required.
Start using it now!

Features

Try the API here

Layout proposto



```
38 dependencies {
39     implementation(libs.activity.ktx)
40     implementation(libs.appcompat)
41     implementation(libs.constraintlayout)
42     implementation(libs.material)
43     /**
44      * Dependências da biblioteca Retrofit2: cliente HTTP e conversor Json
45      */
46     implementation("com.squareup.retrofit2:retrofit:3.0.0")
47     implementation("com.squareup.retrofit2:converter-gson:3.0.0")
48     implementation("com.google.android.gms:play-services-location:21.0.1")
49     testImplementation(libs.junit)
50     androidTestImplementation(libs.espresso.core)
51     androidTestImplementation(libs.ext.junit)
52 }
```



```
AndroidManifest.xml ×
1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:tools="http://schemas.android.com/tools">
4
5      <uses-permission android:name="android.permission.INTERNET" />
6      <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
7      <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
8
9      <application
10         android:allowBackup="true"
11         android:dataExtractionRules="@xml/data_extraction_rules"
12         android:fullBackupContent="@xml/backup_rules"
13         android:icon="@mipmap/ic_launcher"
14         android:label="PrevisaoTempo"
15         android:roundIcon="@mipmap/ic_launcher_round"
16         android:supportsRtl="true"
17         android:theme="@style/Theme.PrevisaoTempo">
```

```
TempoAtual.java x
2
3 import com.google.gson.annotations.SerializedName;
4
5 5 usages
6 public class TempoAtual {
7
8     1 usage
9     @SerializedName("temperature_2m")
10    private double temperature;
11
12     1 usage
13    @SerializedName("relative_humidity_2m")
14    private int relativeHumidity;
15
16     1 usage
17    @SerializedName("wind_speed_10m")
18    private double windSpeed;
19
20    // Getters
21    1 usage
22    public double getTemperature() { return temperature; }
23    1 usage
24    public int getRelativeHumidity() { return relativeHumidity; }
25    1 usage
26    public double getWindSpeed() { return windSpeed; }
27 }
```

```
APIService.java x
1 package api;
2
3 import retrofit2.Call;
4 import retrofit2.http.GET;
5 import retrofit2.http.Query;
6
7 4 usages
8 public interface APIService {
9     1 usage
10     @GET("v1/forecast")
11     Call<HttpResponse> getCurrentWeather(
12         @Query("latitude") double latitude,
13         @Query("longitude") double longitude,
14         @Query("current") String currentParams
15     );
16 }
```

```
© HttpResponseMessage.java x
1 package api;
2
3 import com.google.gson.annotations.SerializedName;
4
5 import models.TempoAtual;
6
7 7 usages
8 public class HttpResponseMessage {
9
10     1 usage
11     @SerializedName("current")
12     private TempoAtual current;
13
14     1 usage
15     public TempoAtual getCurrent() { return current; }
16 }
```

```
© RetrofitClient.java x
3  import retrofit2.Retrofit;
4  import retrofit2.converter.gson.GsonConverterFactory;
5
6  2 usages
7  public class RetrofitClient {
8      1 usage
9      private static final String BASE_URL = "https://api.open-meteo.com/";
10     3 usages
11     private static Retrofit retrofit = null;
12
13     1 usage
14     @ public static ApiService getApiService() {
15         if (retrofit == null) {
16             retrofit = new Retrofit.Builder()
17                 .baseUrl(BASE_URL)
18                 .addConverterFactory(GsonConverterFactory.create())
19                 .build();
20         }
21         return retrofit.create(ApiService.class);
22     }
23 }
```

```
MainActivity.java x
1 package br.com.previsaotempo;
2
3 > import ...
34
35 </> public class MainActivity extends AppCompatActivity {
36
37     7 usages
38     private TextView tvCity;
39     2 usages
40     private TextView tvTemperature;
41     2 usages
42     private TextView tvHumidity;
43     2 usages
44     private TextView tvWindSpeed;
45     2 usages
46     private Button btnRefresh;
47
48     2 usages
49     private FusedLocationProviderClient fusedLocationClient;
50     2 usages
51     private static final int LOCATION_PERMISSION_REQUEST_CODE = 100;
52     1 usage
53     private final String CURRENT_PARAMS = "temperature_2m,relative_humidity_2m,wind_speed_10m";
54 }
```



```
47      @Override
48      protected void onCreate(Bundle savedInstanceState) {
49          super.onCreate(savedInstanceState);
50          setContentView(R.layout.activity_main);
51
52          // Vinculando as Views
53          tvCity = findViewById(R.id.tvCity);
54          tvTemperature = findViewById(R.id.tvTemperature);
55          tvHumidity = findViewById(R.id.tvHumidity);
56          tvWindSpeed = findViewById(R.id.tvWindSpeed);
57          btnRefresh = findViewById(R.id.btnRefresh);
58
59          // Inicializando o cliente de localização
60          fusedLocationClient = LocationServices.getFusedLocationProviderClient(activity: this);
61
62          // Ao abrir o app, tenta pegar a localização
63          checkLocationPermissionAndFetch();
64
65          // Botão de atualizar também pega a localização novamente
66          btnRefresh.setOnClickListener(new View.OnClickListener() {
67              @Override
68              public void onClick(View v) { checkLocationPermissionAndFetch(); }
69          });
70      }
```

```
73
74 // 1. Verifica Permissões
    2 usages
75 private void checkLocationPermissionAndFetch() {
76     if (ContextCompat.checkSelfPermission(context: this,
77         Manifest.permission.ACCESS_FINE_LOCATION) != PackageManager.PERMISSION_GRANTED) {
78         // Solicita a permissão se ainda não foi concedida
79         ActivityCompat.requestPermissions(activity: this, new String[]{Manifest.permission.ACCESS_FINE_LOCATION},
80             LOCATION_PERMISSION_REQUEST_CODE);
81     } else {
82         // Permissão já concedida, busca a localização
83         getLastKnownLocation();
84     }
85 }
86
```

```
87 // 2. Trata a resposta do usuário (Permitiu ou Negou?)
88 @Override
89 public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions,
90                                     @NonNull int[] grantResults) {
91     super.onRequestPermissionsResult(requestCode, permissions, grantResults);
92     if (requestCode == LOCATION_PERMISSION_REQUEST_CODE) {
93         if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
94             getLastKnownLocation();
95         } else {
96             Toast.makeText(context: this, text: "Permissão de localização negada.", Toast.LENGTH_SHORT).show();
97             tvCity.setText("Localização indisponível");
98         }
99     }
100 }
```

```
101
102 // 3. Pega as Coordenadas do GPS
103 // 2 usages
104 private void getLastKnownLocation() {
105     // Verifica novamente a permissão por exigência do Android Studio
106     if (ActivityCompat.checkSelfPermission(context: this,
107         Manifest.permission.ACCESS_FINE_LOCATION) != PackageManager.PERMISSION_GRANTED) {
108         return;
109     }
```

```
110 tvCity.setText("Obtendo localização...");
111
112 fusedLocationClient.getLastLocation().addOnSuccessListener(activity, this, new OnSuccessListener<Location>() {
113     @Override
114     public void onSuccess(Location location) {
115         if (location != null) {
116             double latitude = location.getLatitude();
117             double longitude = location.getLongitude();
118
119             // Descubra o nome da cidade
120             getCityName(latitude, longitude);
121
122             // Busca o clima com as coordenadas dinâmicas
123             fetchWeatherData(latitude, longitude);
124         } else {
125             tvCity.setText("Ligue o GPS do aparelho.");
126         }
127     }
128 });
129 }
130
```

```
131 // 4. Converte Latitude/Longitude em Nome da Cidade (Geocoder)
132 1 usage
133 private void getCityName(double latitude, double longitude) {
134     Geocoder geocoder = new Geocoder(context: this, Locale.getDefault());
135     try {
136         List<Address> addresses = geocoder.getFromLocation(latitude, longitude, maxResults: 1);
137         if (addresses != null && !addresses.isEmpty()) {
138             String cityName = addresses.get(0).getSubAdminArea(); // Pega o nome da cidade
139             if (cityName == null) {
140                 cityName = addresses.get(0).getLocality();
141             }
142             tvCity.setText(cityName);
143         } else {
144             tvCity.setText("Cidade não encontrada");
145         }
146     } catch (IOException e) {
147         e.printStackTrace();
148         tvCity.setText("Erro ao buscar cidade");
149     }
150 }
```

```
151 // 5. Busca os dados na API Open-Meteo
152 1 usage
153 private void fetchWeatherData(double latitude, double longitude) {
154     APIService apiService = RetrofitClient.getApiService();
155     Call<HttpResponse> call = apiService.getCurrentWeather(latitude, longitude, CURRENT_PARAMS);
156
157     call.enqueue(new Callback<HttpResponse>() {
158         @Override
159         public void onResponse(Call<HttpResponse> call, Response<HttpResponse> response) {
160             if (response.isSuccessful() && response.body() != null) {
161                 TempoAtual currentWeather = response.body().getCurrent();
162
163                 tvTemperature.setText(String.format("Temperatura: %.1f °C", currentWeather.getTemperature()));
164                 tvHumidity.setText(String.format("Umidade Relativa: %d %%", currentWeather.getRelativeHumidity()));
165                 tvWindSpeed.setText(String.format("Velocidade do Vento: %.1f km/h", currentWeather.getWindSpeed()));
166             } else {
167                 Toast.makeText(context: MainActivity.this, text: "Erro ao obter clima.", Toast.LENGTH_SHORT).show();
168             }
169         }
170     });
171 }
```



```
170         @Override
171         public void onFailure(Call<HttpResponse> call, Throwable t) {
172             Toast.makeText(context: MainActivity.this, text: "Falha na conexão: " + t.getMessage(),
173                 Toast.LENGTH_SHORT).show();
174         }
175     });
176 }
177
```

```
</> activity_main.xml x
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="match_parent"
4      android:layout_height="match_parent"
5      android:orientation="vertical"
6      android:background="@color/blue"
7      android:gravity="center"
8      android:padding="24dp">
9
10     <TextView
11         android:id="@+id/tvTitle"
12         android:layout_width="wrap_content"
13         android:layout_height="wrap_content"
14         android:text="Clima Atual"
15         android:textSize="28sp"
16         android:textStyle="bold"
17         android:layout_marginBottom="32dp"/>
18
```

```
19 <TextView
20     android:id="@+id/tvCity"
21     android:layout_width="wrap_content"
22     android:layout_height="wrap_content"
23     android:text="Buscando localização..."
24     android:textSize="22sp"
25     android:textColor="#3F51B5"
26     android:layout_marginBottom="24dp"/>
27
28 <TextView
29     android:id="@+id/tvTemperature"
30     android:layout_width="wrap_content"
31     android:lay Salvo neste PC ht="wrap_content"
32     android:text="Temperatura: -- °C"
33     android:textSize="20sp"
34     android:layout_marginBottom="16dp"/>
35
36 <TextView
37     android:id="@+id/tvHumidity"
38     android:layout_width="wrap_content"
39     android:layout_height="wrap_content"
40     android:text="Umidade Relativa: -- %"
41     android:textSize="20sp"
42     android:layout_marginBottom="16dp"/>
```

```
43
44     <TextView
45         android:id="@+id/tvWindSpeed"
46         android:layout_width="wrap_content"
47         android:layout_height="wrap_content"
48         android:text="Velocidade do Vento: -- km/h"
49         android:textSize="20sp"
50         android:layout_marginBottom="40dp"/>
51
52     <Button
53         android:id="@+id/btnRefresh"
54         android:layout_width="wrap_content"
55         android:layout_height="wrap_content"
56         android:text="Atualizar Dados"/>
57
58 </LinearLayout>
```

